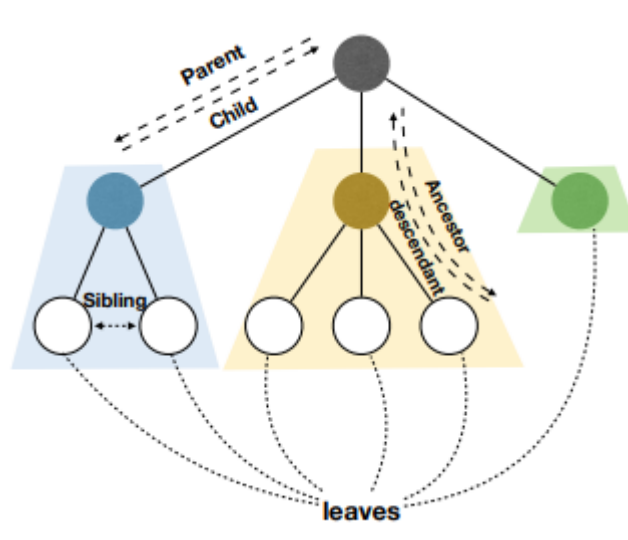


Tree

1. Definition

树是一个连通的无环无向图。

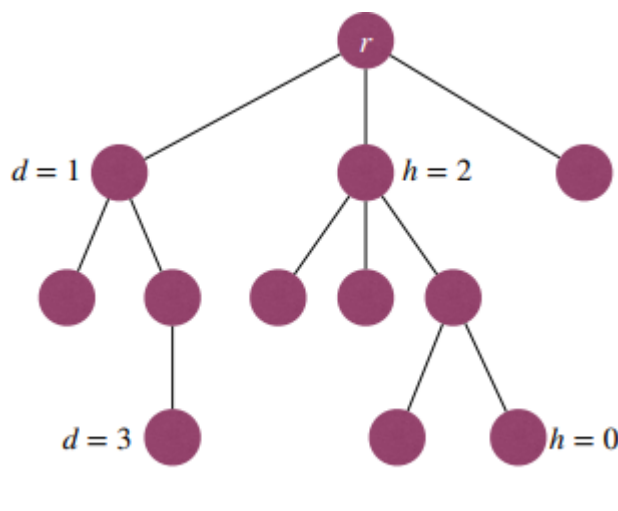


2. More terminology on Trees

节点 u 的深度是从 u 到根 r 的路径长度

节点 u 的高度是从 u 到它的任一后代中最长的路径

- 叶节点的高度为0
- 非叶节点的高度是它孩子的最大高度加1



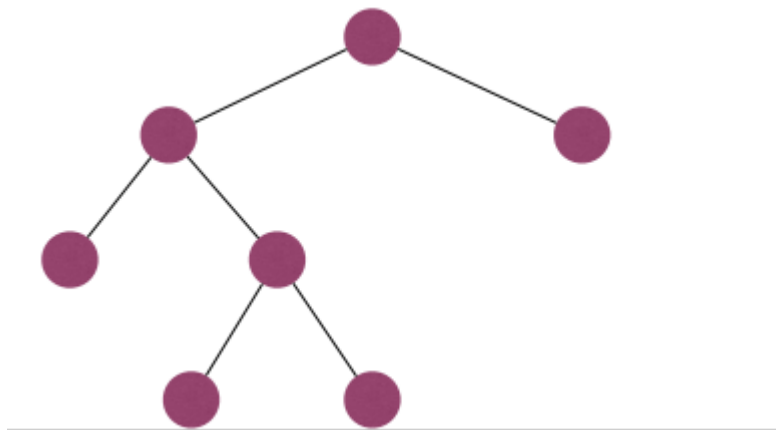
3. Some Trees

3.1 Binary Trees(二叉树)

二叉树的每个节点最多有两个孩子，通常称呼为左孩子和右孩子

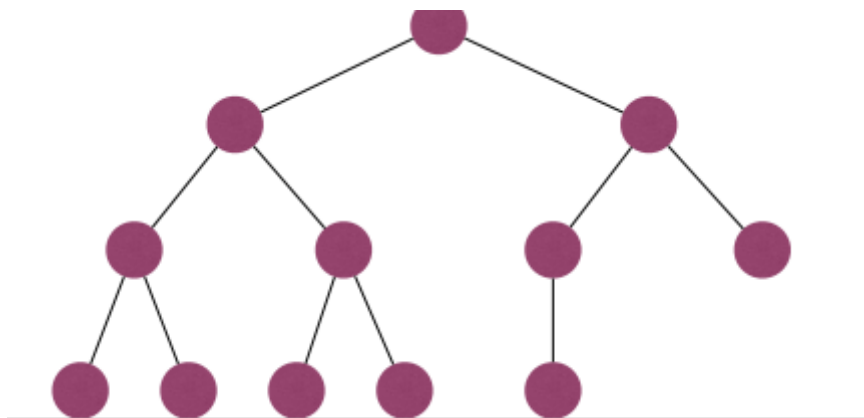
3.2 Full Binary Trees(满二叉树)

满二叉树的节点除了叶子外都有两个孩子



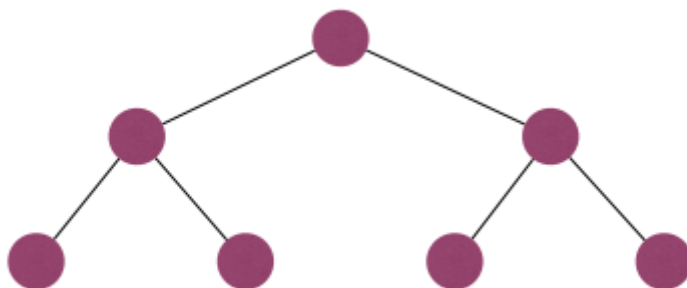
3.3 Complete Binary Trees(完全二叉树)

完全二叉树的每一层除了最后一层都是满的，而最后一层的所有节点都尽可能的向左



3.4 Perfect Binary Trees(完美二叉树)

完美二叉树的所有非叶子节点都有两个孩子并且所有的叶子深度都是相同的



4.Represent Binary Trees

```
1 class Node{
2     Data data
3     Node parent
4     Node left
5     Node right
6 }
```

5.Tree Traversals

5.1 Preorder traversal(先序遍历)

先访问根节点，再访问左子树，最后访问右子树

```
1 PreorderTrav(r):  
2   if r != NULL  
3     Visit(r)  
4     for each child u of r  
5       PreorderTrav(u)
```

5.2 Postorder traversal(后序遍历)

先左子树，再右子树，最后根节点

```
1 PostorderTrav(r):  
2   if r != NULL  
3     for each child u of r  
4       PostorderTrav(u)  
5     Visit(r)
```

5.3 Inorder traversal(中序遍历)

先左子树，再根节点，最后右子树

```
1 InorderTrav(r):  
2   if r != NULL  
3     InorderTrav(r.left)  
4     Visit(r)  
5     InorderTrav(r.right)
```

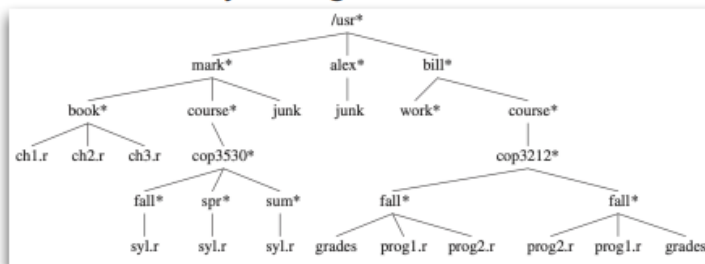
5.4 Complexity of recursive traversal

时间复杂度为 $\Theta(n)$

空间复杂度: $O(n)$ as worst-case **call stack** depth is $\Theta(n)$

5.5 Sample application of preorder traversal

• Directory Listing



ListDir(obj, depth):

if *obj* $\neq$ NULL

 PrintName(*obj*, *depth*)

if IsDirectory(*obj*)

for each subobj **in** *obj*

 ListDir(subobj, *depth* + 1)

```
/usr
mark
  book
    ch1.r
    ch2.r
    ch3.r
  course
    cop3530
      fall
        syl.r
      spr
        syl.r
      sum
        syl.r
  junk
alex
  junk
bill
  work
  course
    cop3212
      fall
        grades
        prog1.r
        prog2.r
      fall
        prog2.r
        prog1.r
        grades
```

5.6 Iterative tree traversal

用栈来进行递归

```
1  class Frame{
2      Node node
3      bool visit
4      Frame(Node n, bool v){
5          node = n
6          visit = v
7      }
8  }
9
10
11 PreorderTravIter(r):
12     Stack s
13     s.push(Frame(r, false))
14     while !s.empty()
15         f = s.pop()
16         if f.node != NULL
17             if f.visit
18                 visit(f.node)
19             else
20                 for each child u of f.node
21                     s.push(Frame(u, false))
22                     s.push(Frame(f.node, true))
23
24 PostorderTravIter(r):
25     Stack s
26     s.push(Frame(r, false))
27     while !s.empty()
28         f = s.pop()
29         if f.node != NULL
30             if f.visit
31                 visit(f.node)
```

```

32         else
33             for each child u of f.node
34                 s.push(Frame(f.node,true))
35                 s.push(Frame(u,false))
36
37 InorderTravIter(r):
38     Stack s
39     s.push(Frame(r,false))
40     while !s.empty()
41         f = s.pop()
42         if f.node != NULL
43             if f.visit
44                 Visit(f.node)
45             else
46                 s.push(Frame(f.node.right,false))
47                 s.push(Frame(f.node,true))
48                 s.push(Frame(f.node.left,false))
49

```

时间复杂度为 $\Theta(n)$

空间复杂度为 $O(n)$

5.7 Level-order traversal of trees(层序遍历)

```

1 LevelorderTrav(r):
2     if r != NULL
3         Queue q
4         q.add(r)
5         while !q.empty()
6             node = q.remove()
7             if node != NULL
8                 Visit(node)
9                 q.add(node.left)
10                q.add(node.right)

```

时间复杂度为 $\Theta(n)$

空间复杂度为 $\Theta(n)$ 在最坏的情况下

5.8 Morris Traversal(神级遍历)

- 实质：建立一种机制，对于没有左子树的节点只到达一次，对于有左子树的节点会到达两次
- 记作当前节点为cur
 - 1.如果cur无左孩子，cur向右移动 (cur=cur.right)
 - 2.如果cur有左孩子，找到cur左子树上最右的节点，记为mostright
 - 1.如果mostright的right指针指向空，让其指向cur，cur向左移动 (cur=cur.left)
 - 2.如果mostright的right指针指向cur，让其指向空，cur向右移动 (cur=cur.right)